

Tech Biz Security Ethical Hacking Internship

Week 4 · Task 11 — How the Web Works + Burp Suite Intro

Submitted by: Mah Noor

Intern Name	Mah Noor
Username (THM)	mahnnorchaudary
Program	Tech Biz Security — Ethical Hacking Track
Task	Week 4 · Task 11 — How the Web Works + Burp Suite Intro
Submission Date	September 13, 2026

1. TryHackMe — Room Status

The assigned room "Burp Suite: The Basics" requires a TryHackMe Premium subscription and was not accessible on the free tier. A screenshot of the locked room is provided below as proof of attempt. As instructed in the task guidelines, the practical work was completed on DVWA (local environment) as an alternative.



Figure 1 — Burp Suite: The Basics is a Premium room (locked on free tier)

2. HTTP Request & Response — Fundamentals

2.1 What is HTTP/HTTPS?

HTTP (HyperText Transfer Protocol) is the foundation of data communication on the web. Every time a browser loads a page or an app calls an API, it sends an HTTP request and receives an HTTP response. HTTPS is the encrypted version of HTTP — it wraps the same request/response structure inside a TLS layer to prevent interception.

2.2 Structure of an HTTP Request

- Request Line: Method + Path + HTTP version — e.g. GET /api/cashback/users/cashback-user HTTP/2
- Headers: Key-value pairs — Host, Cookie, Content-Type, User-Agent, etc.

- Body: Present in POST/PUT requests; contains form data or JSON payload

2.3 Structure of an HTTP Response

- Status Line: HTTP version + status code + reason — e.g. HTTP/2 200 OK
- Headers: Content-Type, Content-Length, Set-Cookie, Cache-Control, security headers
- Body: The actual content returned — HTML, JSON, images, etc.

2.4 Common HTTP Status Codes

- 200 OK — Request succeeded
- 301/302 — Redirect (permanent/temporary)
- 400 Bad Request — Malformed request syntax
- 401 Unauthorized — Authentication required
- 403 Forbidden — Access denied
- 404 Not Found — Resource does not exist
- 500 Internal Server Error — Server-side failure

2.5 Cookies & Sessions

Cookies are small pieces of data that the server sends to the browser (via Set-Cookie header) and the browser sends back on every subsequent request (via Cookie header). They are used to maintain sessions — since HTTP is stateless, cookies allow the server to recognize a returning user without requiring them to log in on every request.

- Session Cookie: Temporary, deleted when browser closes
- Persistent Cookie: Has an expiry date (Expires/Max-Age attribute)
- Security flags: HttpOnly (not accessible to JS), Secure (HTTPS only), SameSite (CSRF protection)

3. Burp Suite — Request Interception

3.1 What is Burp Suite?

Burp Suite is a web application security testing tool. It acts as a proxy between the browser and the server — intercepting, inspecting, and modifying HTTP/S traffic in real time. In this task, the proxy was configured in the browser, and a live request was captured and analyzed.

3.2 Intercepted Request Analysis

The screenshot below shows a real GET request intercepted in Burp Suite, sent to cashback.opera.com. The request and response were analyzed to understand HTTP structure in practice.

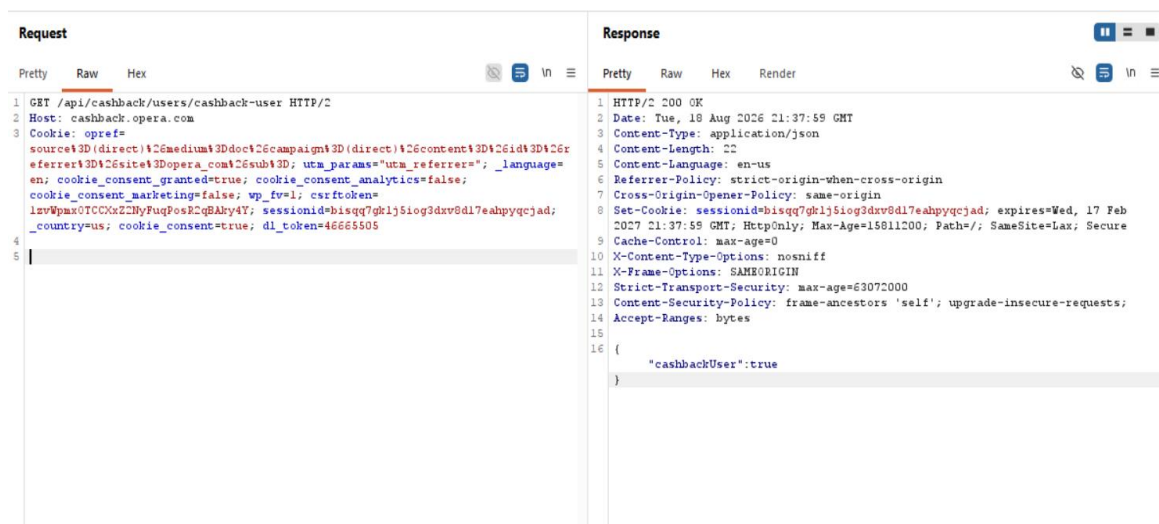


Figure 2 — Burp Suite intercept: GET /api/cashback/users/cashback-user HTTP/2 with full request/response

Component	Description
Request Line	Method + URL + HTTP version (e.g. GET /api/cashback/users/cashback-user HTTP/2)
Host Header	Target domain — cashback.opera.com
Cookie	Session tokens, consent flags, CSRF token — sent automatically with each request
Status Code	200 OK — request succeeded, server returned JSON body
Response Body	<code>{"cashbackUser": true}</code> — JSON confirms user is enrolled in cashback program
Set-Cookie	Server refreshes sessionid with HttpOnly; Secure; SameSite=Lax flags

3.3 Security Headers Observed in Response

- Strict-Transport-Security: max-age=63072000 — Forces HTTPS for 2 years
- X-Content-Type-Options: nosniff — Prevents MIME-type sniffing attacks
- X-Frame-Options: SAMEORIGIN — Prevents clickjacking via iframes
- Content-Security-Policy — Restricts resource loading origins
- SameSite=Lax on session cookie — Protects against CSRF

4. GET vs POST — Key Differences

Aspect	GET	POST
Data location	URL query string (?key=value)	Request body (hidden from URL)
Use case	Fetching data, search queries	Login forms, uploading data
Security	Parameters visible in browser history/logs	Body not in URL — slightly more private
DVWA example	Brute Force page: <code>?username=admin&password=pass</code>	Login page: credentials sent in request body

In DVWA, the Brute Force module uses a GET request — credentials appear directly in the URL bar (?username=admin&Login=Login), making them easy to intercept. The main login form uses POST — credentials are in the body, not the URL, which is slightly safer (though still not encrypted unless HTTPS is used).

5. DVWA Setup on Windows (XAMPP)

5.1 Environment

- OS: Windows
- Web Server: Apache via XAMPP v3.3.0 (running on ports 80, 443)
- PHP Version: 8.0.30
- Database: MySQL (configured in DVWA config.inc.php)
- Security Level set to: Low

5.2 Setup Process & Screenshots

Step 1 — XAMPP Apache Running

Apache was started via XAMPP Control Panel. MySQL showed a startup error (blocked port / missing dependencies), but Apache remained running on ports 80 and 443, which is sufficient for DVWA.

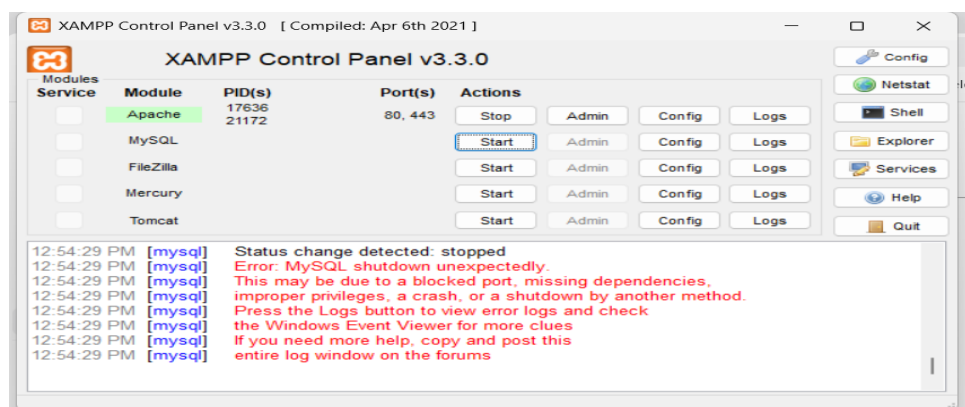


Figure 3 — XAMPP Control Panel: Apache running on ports 80 & 443

Step 2 — Initial 404 (DVWA not found)

First attempt at localhost/DVWA/setup.php returned 404 — DVWA files were not yet placed in the correct htdocs directory. This was resolved by copying the DVWA folder into C:\xampp\htdocs\.

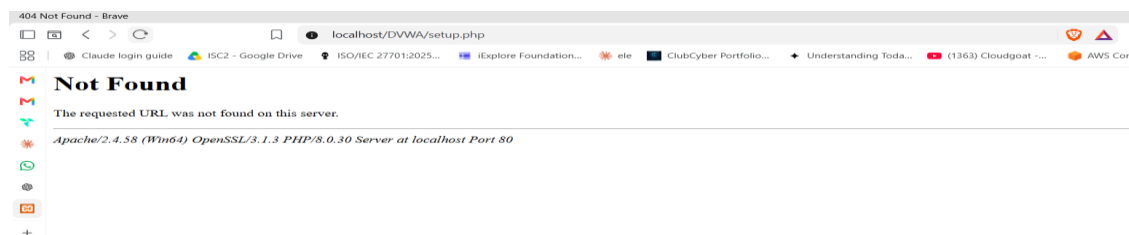


Figure 4 — 404 Not Found before placing DVWA in htdocs

Step 3 — DVWA Setup Page

After correct placement, setup.php loaded successfully. Setup Check confirmed: Apache running, PHP 8.0.30, mod_rewrite enabled, writable folders confirmed. reCAPTCHA key was missing (not required for basic labs) and PHP module gd was missing (only needed for CAPTCHA labs).

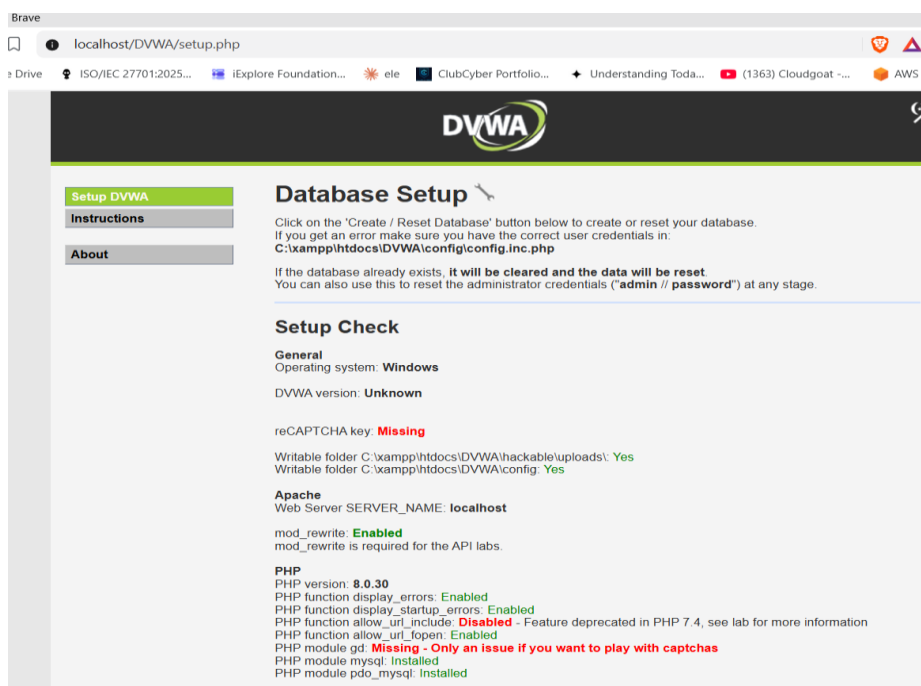


Figure 5 — DVWA Setup Check: Apache & PHP confirmed, database ready to create

Step 4 — DVWA Home Page

Database was created successfully. DVWA loaded at localhost/DVWA/index.php with all vulnerability modules visible: Brute Force, Command Injection, CSRF, File Inclusion, SQL Injection, XSS (DOM/Reflected/Stored), and more.

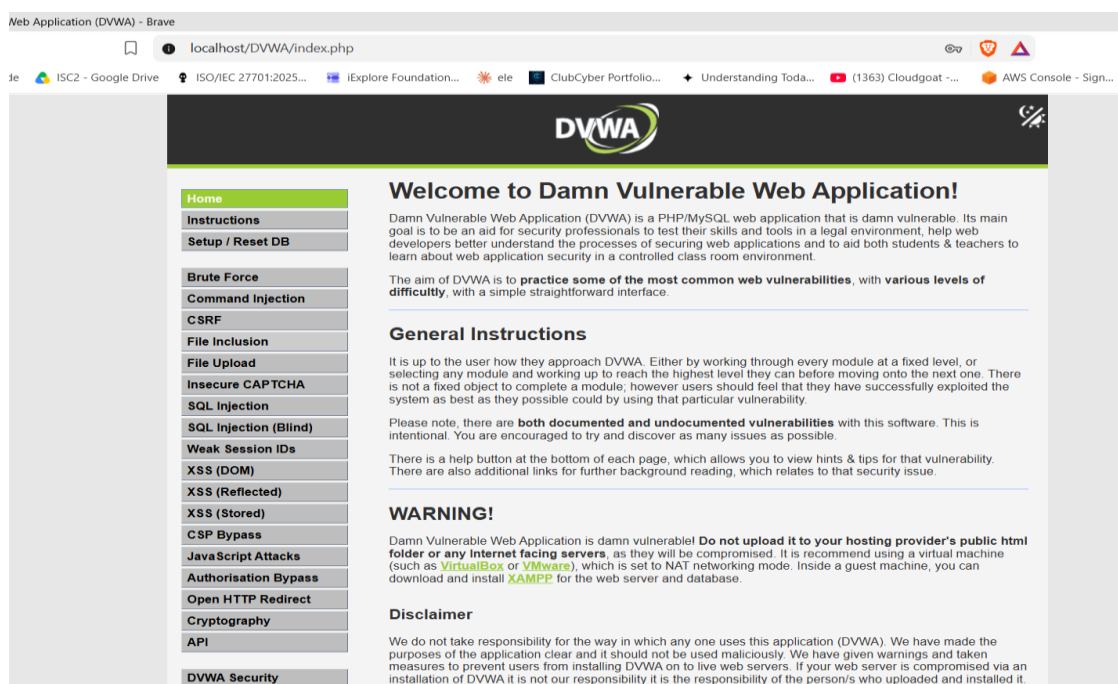


Figure 6 — DVWA Home Page: all vulnerability modules accessible

Step 5 — Security Level Set to Low

DVWA Security was set to "Low" to allow full exploitation during practice. At Low level, no security measures are in place — ideal for learning basic attack techniques.

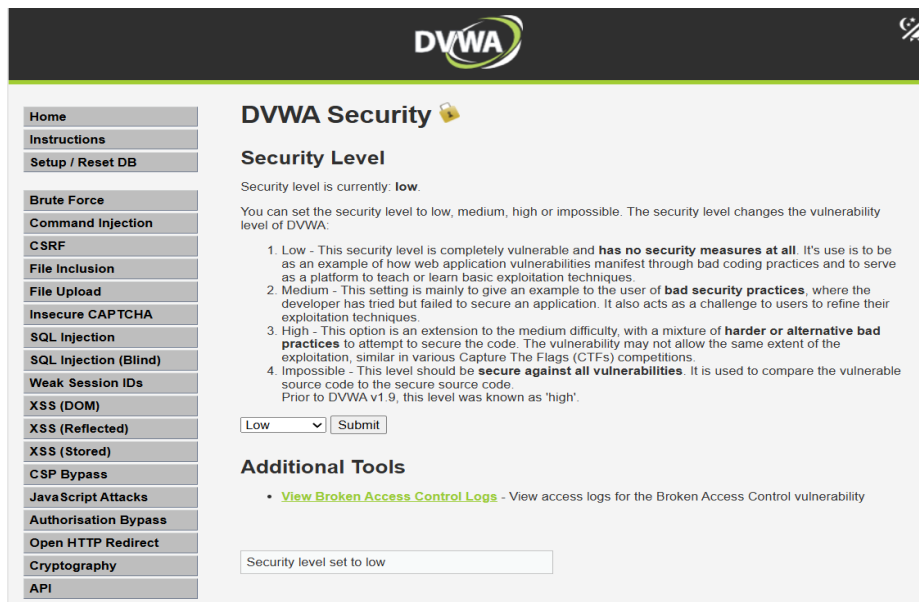


Figure 7 — DVWA Security Level set to Low

6. Key Takeaways

- HTTP is stateless — cookies/sessions maintain user state across requests
- Every HTTP request has a method, headers, and optionally a body; every response has a status code, headers, and body
- Burp Suite intercepts live traffic — security headers, cookies, and response data are all visible and modifiable
- GET exposes parameters in the URL; POST hides them in the body — but neither is truly secure without HTTPS
- DVWA provides a safe, legal environment to practice all major web vulnerabilities
- Security headers (HSTS, X-Frame-Options, CSP, SameSite) are the server's defense layer against common attacks